



University of Zurich^{UZH}

High Performance Computing Lecture 14

Douglas Potter

Cesare Cozza

Mark Eberlein

Examination

Date:

Wednesday

January 14th

13:00 to 15:00

Y16-G-15

Materials allowed:

- Course Slide
- Personal notes
- Read with Laptop/Tablet

Compilers and Queues

- Version control (e.g. “git”): clone, pull, add, commit, push.
- Phases of a compiler (compile, link).
- Importance of compiler optimization and how can affect the results.
- Basics of “make”
- Purpose and function of batch queue systems (e.g. SLURM). Scheduling: priority versus FIFO. How you request resources (nodes, cores, etc.)

Performance

- Moore's Law and how it relates to the Top500 list
- Strong and weak scaling
- Latency and bandwidth.
- Memory, cache and bus (e.g., attached network or GPU card) hierarchy and their relative performance.
- Different ways of instrumenting (benchmarking) your code.

OpenMP

- Directive based
- Model (shared memory, threads, a “thread” runs on a “core”)
- The serial and “parallel” (region) parts.
- The “parallel for” loop.
- Synchronization between threads, for example “reduce” clause, or “atomic” or “critical” pragmas. Performance of each.
- Shared versus private variables.
- How OpenMP schedules work between threads.

MPI

- Model (distributed memory, message passing, a “rank” is a process).
- How to split work between “ranks” (also known as load balancing or domain decomposition).
- How to compile MPI programs (e.g., mpicc, cc, mpicxx, CC, etc.) — “wrappers”.
- Some of the common functions (those covered in the lectures).
- How message passing works and common problems (e.g., deadlocks).

Hybrid Computing

- CPU “sockets” and “cores”, and GPU “SMs” and “cores”.
- SIMD (AVX) on CPUs and “Warps” on GPUs.
- CPU versus GPU
 - CPU: small number of high performance cores and ~ one thread per core.
 - GPU: large number of lower performance cores and many threads per core.
 - Memory bandwidth to memory on each.
- Divergence: how it is handled on the CPU (SIMD) and on the GPU (Warps).
- Data alignment: what it is and why is it important.
- Latency hiding and Occupancy on GPUs. Latency to start kernels or data transfer. Latency of instruction on the GPU versus the CPU.

OpenACC

- Directive Based
- Basic GPU operations: allocate, copy, kernel launch.
- Data management: how OpenACC gets your data to where it needs to be (GPU or CPU) and how you can steer this with a “data” construct.
- Difference between “kernels” construct and “parallel” construct.
- Difference between “grid”, “worker” and “vector”.
- Shared versus private variables.
- Synchronization on the GPU, for example “reduce” clause, or “atomic” or “critical” pragmas. Performance of each.
- What asynchronous operations do and way you would want to use them.

CUDA

- How to compile CUDA code (nvcc).
- What a streaming multiprocessor (SM) is.
- What is a “grid”, “block”, “warp” and “thread” and how they relate to the SM.
- How indexing works. How to turn a thread and block index into a global index.

Cloud & Containers

- Difference between Cloud (Virtual Machines) and containers
 - VM: a “virtual” computer. Has memory, CPUs, network, disks, etc.
- How to handle “persistence” with VMs and containers, e.g., “snapshots”.
- Ephemeral computing (create a resource, compute, throw away the resource).
- You are “root” in virtual machines and containers and what this means.
- Dockerhub: like github but for containers.
- How to access data
 - images and snapshots for VMs; mounting host directories for containers.

MapReduce

- Why the “compute” is sent to the “data” instead of the normal “HPC” way.
- Data model: write once and read (process) multiple times.
- What the “map” and “reduce” phases do.
- What are the “key” and “value”?



ESC412 Advanced HPC

- Only “exercises”
 - Development of a HPC application
 - Incremental – performance improved each week
 - Answer given each week so you can keep progressing
- Mandatory for the Computational Science Master
 - If taking this program you may want to take this course during the Master

Beyond Von Neumann



MPI and OpenMP



MPI+X



HPC Architectures



Exascale challenges



Task Based Computing



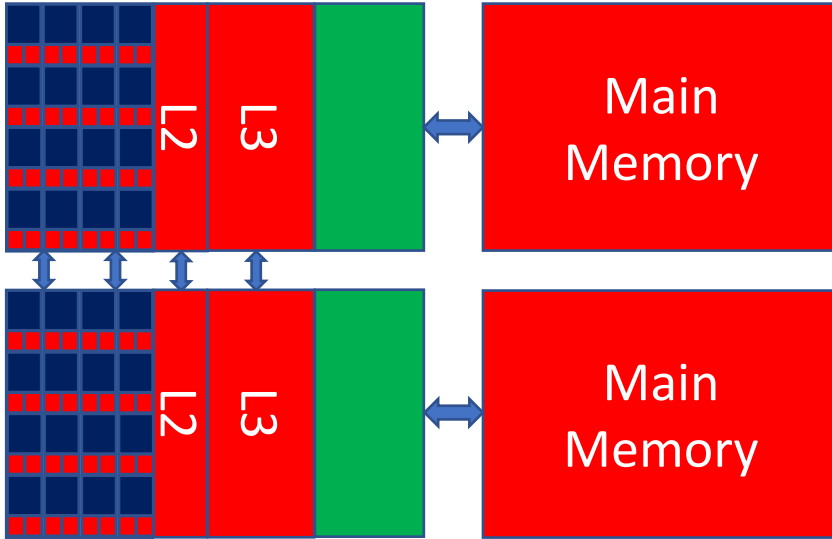
The NeoDigital Age

Neuromorphic

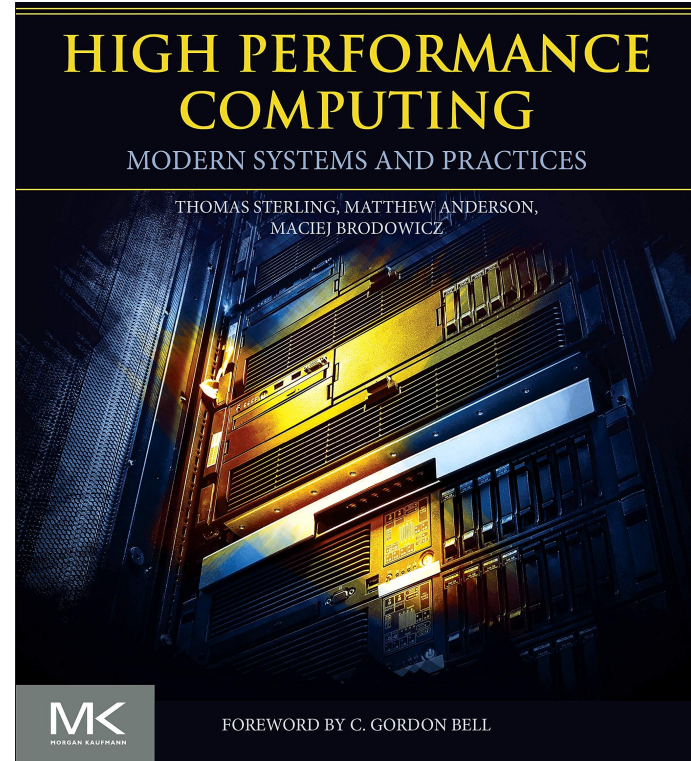
Quantum Computing

Future trends in High Performance Computing

Von Neumann



Chapter 21:



Message Passing Interface (MPI)

- We discussed only a few of the more than 100 MPI-1 functions.
- MPI continues to be developed
- MPI-2 added several important new features.
 - A set of I/O functions to optimize parallel access to storage.
 - Single sided function to directly access remote memory
 - Similar to, but **NOT** shared memory. Many constraints and still “messages”.
- MPI-3 added non-blocking collectives.

OpenMP

- We discussed only the most important aspects
 - Parallel regions
 - Parallel “for” loops
- OpenMP version 3 introduced the concept of multitasking
 - the “task” construct
- Version 4 expands even further
 - Support for heterogeneous systems including GPUs or other accelerators
 - Thread affinity
 - Support for vector operations (SIMD/AVX for example)

MPI+X

- MPI is a reliable, stable and mature tool for coarse-grained parallelism across distributed memory machines.
- OpenMP provides finer grained parallelism on a single shared-memory machine.
- MPI+OpenMP are often used together.
- MPI+X describes MPI and something else in general.

The “end” of Moore’s law?

- For many decades we saw an **exponential** growth in the number of transistors on a device, and of its clock speed.
- For the last few years, the single core performance has remained essentially unchanged.
- The exponential increase in performance continues, perhaps slower, but now it is driven by the number of cores on a chip.
- The only way to continue this growth seems to be by increasing the parallelism of the systems: more cores per node, vectorization of the cores, and/or more nodes.

The road to Exascale

Piz Daint (Switzerland)

- 21 Pflops
- 2.4 mW; 10.4 Gflops/Watt
- 5,704 nodes
- CISC processors + GPUs
- 68,448 CPU cores + 5,704 GPUs
 - 361,760 “cores”
- 0.365 PB of memory

Sunway TaihuLight (China, 2016)

- 125 Pflops
- 15.4 mW; 6.1 Gflops/Watt
- 40,960 nodes
- RISC processors
- 10,649,600 cores
- 1.3 PB of memory

The road to Exascale

Piz Daint (Switzerland)

- 21 Pflops
- 2.4 mW; 10.4 Gflops/Watt
- 5,704 nodes
- CISC processors + GPUs
- 68,448 CPU cores + 5,704 GPUs
 - 361,760 “cores”
- 0.365 PB of memory

Summit (USA, 2018)

- 149 Pflops
- 10.1 mW; 14.7 Gflops/Watt
- 4,608 nodes
- CISC processors + GPUs
- 202,752 CPU cores + 27,648 GPUs
 - 2,414,592 “cores”
- 2.8 PB of memory

The road to Exascale

Piz Daint (Switzerland)

- 21 Pflops
- 2.4 mW; 10.4 Gflops/Watt
- 5,704 nodes
- CISC processors + GPUs
- 68,448 CPU cores + 5,704 GPUs
 - 361,760 “cores”
- 0.365 PB of memory

Fugaku (Japan, 2020)

- 442 Pflops
- 29.9 mW; 15.4 Gflops/Watt
- 158,976 nodes
- ARM processors
- 7,630,848 cores
- 5.1 PB of memory

The road to Exascale

Piz Daint (Switzerland)

- 21 Pflops
- 2.4 mW; 10.4 Gflops/Watt
- 5,704 nodes
- CISC processors + GPUs
- 68,448 CPU cores + 5,704 GPUs
 - 361,760 “cores”
- 0.365 PB of memory

Frontier (USA, June 2022)

- 1,102 Pflops
- 21.1 mW; 52.2 Gflops/Watt
- 9,248 nodes
- AMD Epyc CPU + 4 AMD GPUs
- 591,872 CPU cores + 36,992 GPUs
 - 8,730,112 “cores”
- 4.5 PB of memory

The road to Exascale

- Might be a trend towards “lightweight” architectures
 - Instead of a small number of powerful nodes
 - A large number of more power efficient nodes (slower clock, more cores)
- Increasingly heterogeneous
 - GPU in addition to CPU on nodes (probably more than one)
- TaihuLight nodes as well as the Xeon Phi (now discontinued)
 - GPU-like memory management required, but CPU-like software
- ARM architecture (Mont Blanc, Alps)
- Field Programmable Gate Arrays (FPGAs)

Exascale Challenges

- Energy and power. Power costs CHF 1M per Mwatt per year.
 - Efficiency needs to reach something like 50 Gflops per Watt – Frontier
- Hardware parallelism – billion-way
 - One billion cores – not the Frontier way though
 - 100 million cores each with 10 way SIMD like vector operations.
- Software challenge: support multi-level, billion-way parallelism.
- Latency: larger systems lead to longer connections.
- Reliability: mean time between failure (MTBF).

Asynchronous Multitasking

- Current models (e.g., MPI and OpenMP) use **static** work scheduling.
 - Synchronization is **expensive**.
- Moving towards “asynchronous multitasking”.
- Work is decoupled from each other (made independent).
- Scheduling is done **dynamically**
 - When a piece of the work finishes it frees a computing resource.
 - That resource takes the next available work package
 - Work is scheduled on the most appropriate resource (memory, CPU, GPU)

Multithreaded Applications

- This is when more than one thread operates concurrently
- We've already seen this! OpenMP is multithreaded.
- Traditionally we assign a single thread to each compute “core”.
- This is not the only way: we can launch more threads than cores.
 - For example 1000 threads for 10 cores.
 - The operate independently and are scheduled on the cores.
- This practice is known as “overdecomposition”
- We saw this with GPU devices when we used CUDA and OpenACC.
- Thread switching is not always so efficient.

Message Driven Computation

- The standard approach is a load/store architecture
 - Data is stored in main memory, perhaps on a remote node.
 - To process it we read it into the CPU (and maybe over the network).
 - The results are stored back in main memory.
 - Latency and bandwidth problems are addressed with a variety of caches.
- An alternative approach is to send the “code” to the data
 - Remote Procedure Calls
 - ParallelX and HPX are more modern versions of this
 - Same principle as Hadoop and MapReduce!

Task-Based Parallelism

- Actor Synchronization
 - Dataflow and “futures”
 - Essentially we apply enforce an ordering on the calculations
 - Avoid course-grained synchronization like barriers or collectives.
- Runtime System Software
 - Charm++ or HPX for example
 - Create a dependency graph of all of the calculations (a DAG)
 - Runtime system schedules the work and data movement
 - Truly dynamic and asynchronous
 - **High overhead**, but often excellent strong **and weak** scaling.

The NeoDigital Age

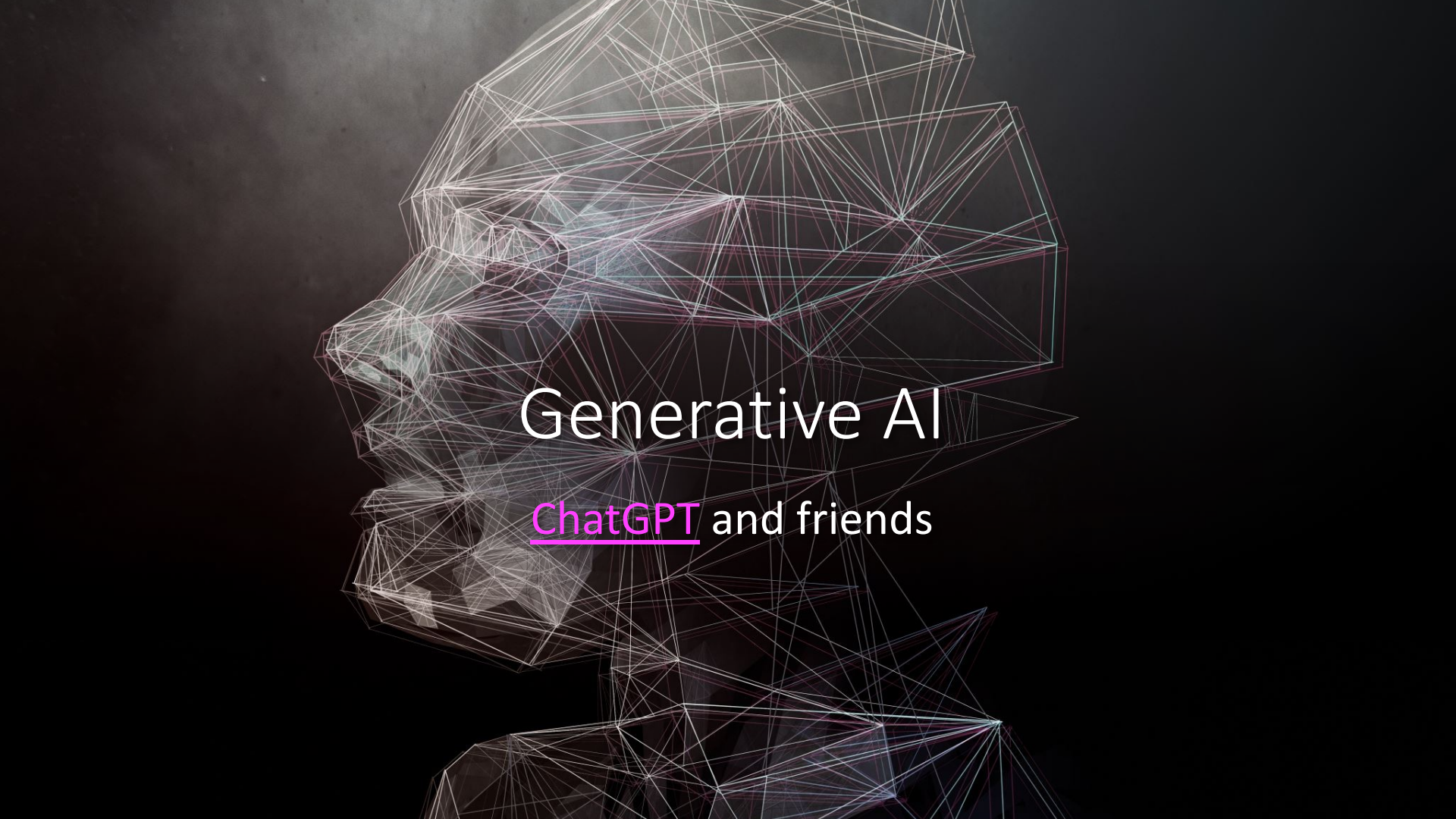
- Right now we take the Von Neumann approach and parallelize it
 - Some fundamental shift will be required to pass from Exascale to Zetascale
- Memory access rather than floating point performance limits us
- New algorithms have the potential to vastly advance certain fields
- Programming interfaces will evolve
 - MPI-1 to MPI-4
 - The introduction of OpenMP
 - CUDA
 - What next?

Neuromorphic Computing

- The human brain has 89 billion neurons but consumes only 25 Watts
 - Remember that supercomputers consume 25 million Watts
- The brain operates at 1 kHz compared to GHz.
- Idea is to mimic the brain
 - Definitely not Von Neumann
 - Neurons store, compute and communicate
- Artificial Neural Networks (ANN) try to mimic this
 - Should be very good for AI and machine learning
 - Programming will be a real challenge
 - Lower energy and faster than Von Neumann approach.

Quantum Computing

- Quantum Computing is complicated
- Standard Digital Computers store “bits” that are zero or one.
 - This bits are manipulated and stored.
- Quantum computers use a “qubit”.
 - They exist in the quantum world – a world of probabilities.
 - A “qubit” is not zero or one, it is **both** zero and one.
 - If you have “n” qubits you can store all 2^n possible values simultaneously.
- Quantum algorithms can simultaneously solve all
 - Shor’s algorithm can factor large prime numbers

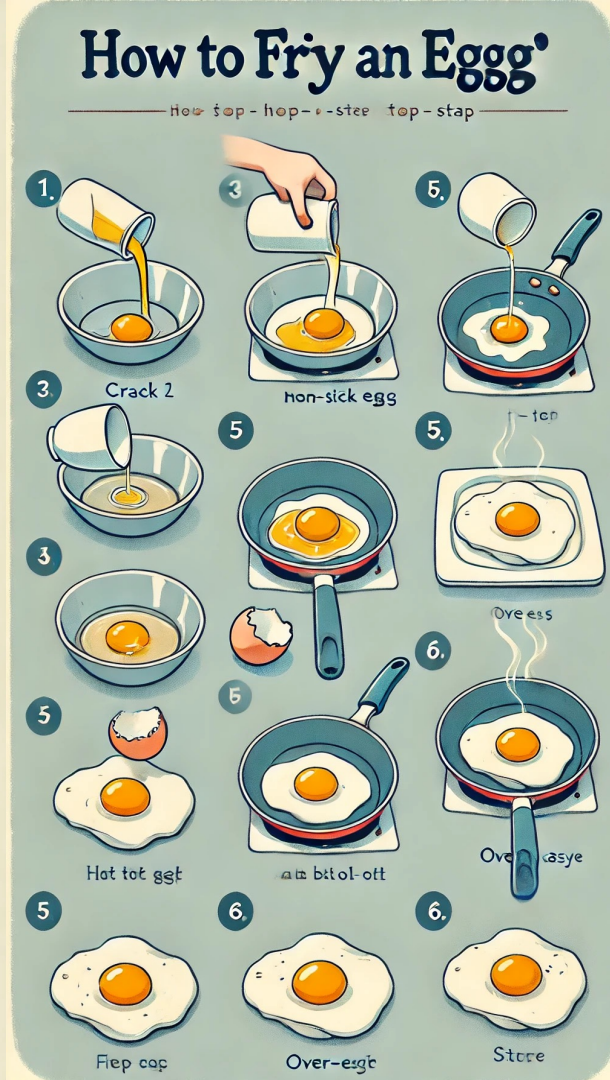
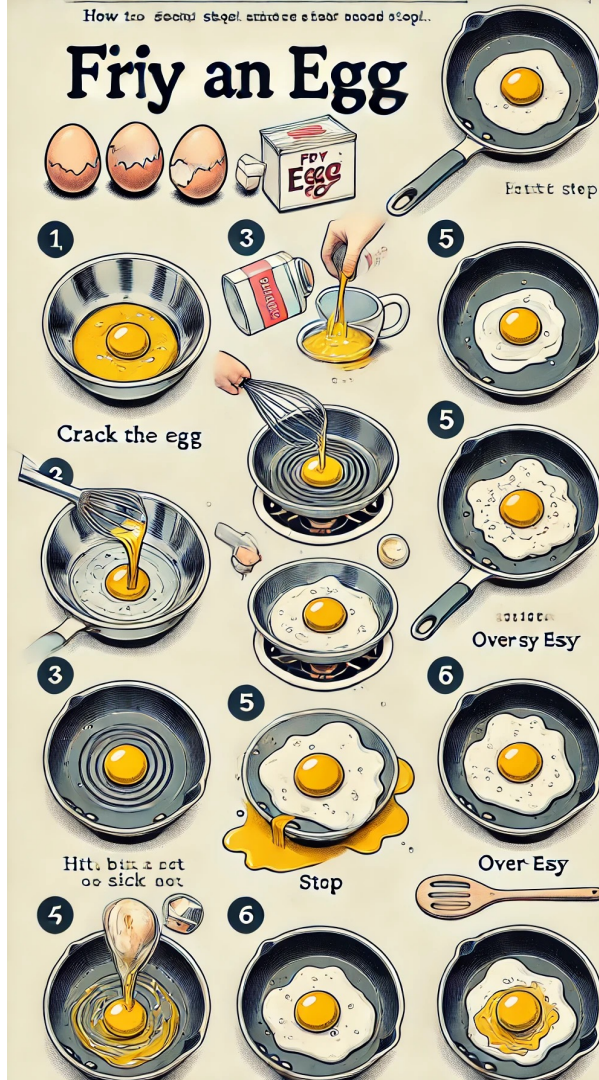
An abstract, wireframe-style illustration of a human head in profile, facing left. The head is composed of a complex network of thin, white lines that form a mesh-like structure. The lines are interconnected to create a sense of depth and volume, with some lines appearing more prominent than others. The background is a dark, textured gray, and the overall aesthetic is futuristic and digital.

Generative AI

ChatGPT and friends

How to fry an egg

Here are two visual charts showing the step-by-step process of frying an egg. Let me know if you need any adjustments or additional details!



Evaluating the Code Quality of AI-Assisted Code Generation Tools: An Empirical Study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT

Burak Yetiştiren, Işık Özsoy, Miray Ayerdem, Eray Tüzün, 2023

- ChatGPT, GitHub Copilot, and Amazon CodeWhisperer generate correct code 65.2%, 46.3%, and 31.1% of the time
- Developers write code 55% faster when using Copilot
 - junior developers use Copilot about 20% more than more experienced developers
- Generated code is more opaque
 - Results in copy/paste like code
 - More code is reverted (double the rate)
 - Quality is lower (more security issues)
- Great for
 - API quick reference
 - Double checking written code
 - Exploration – is there a better way to write this
- Still needs an “expert” to verify and adjust